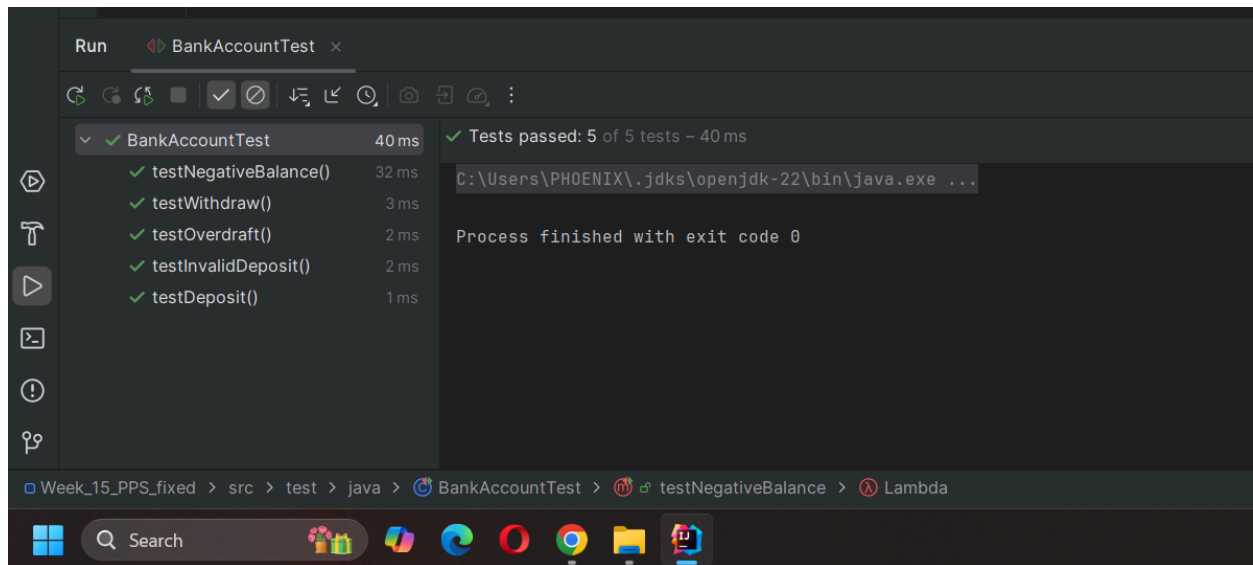


The prompts and answers a-c:

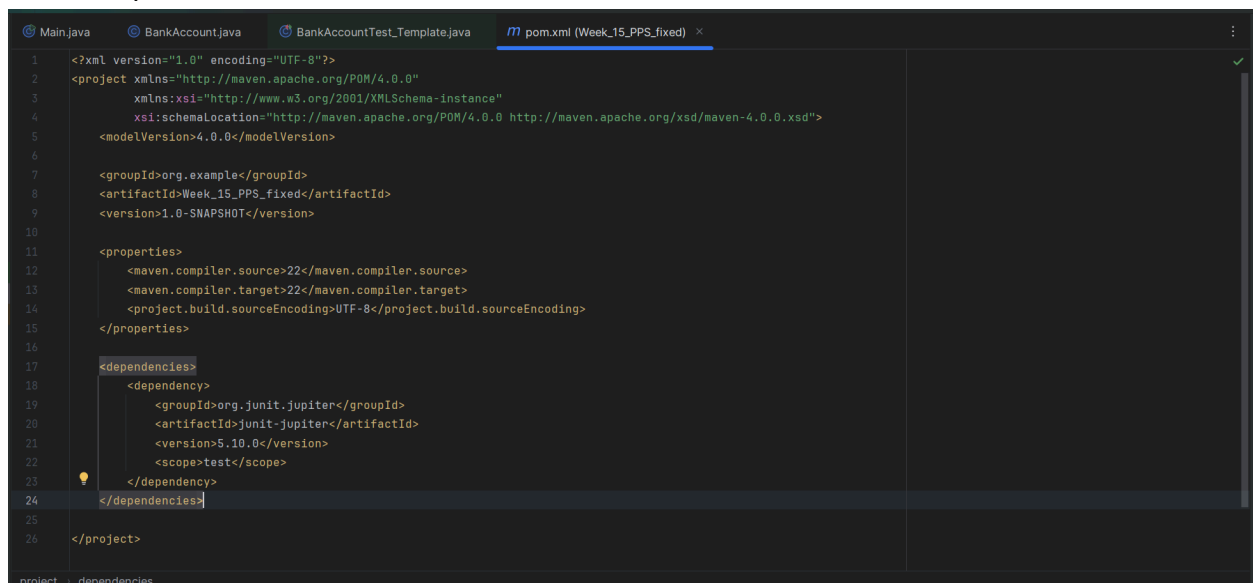
```
Main.java  BankAccount.java  BankAccountTest_Template.java  pom.xml (Week_15_PPS_fixed)

10
11
12 class BankAccountTest {
13
14     private BankAccount account; 9 usages
15
16     @BeforeEach
17     void setUp() {
18         // Starts each test with a fresh account of 100.0
19         account = new BankAccount( initialBalance: 100.0);
20     }
21
22     /** 1. Add an @AfterEach annotation and method to delete the current bank account to make it available for garbage collection */
23     @AfterEach
24     void tearDown() {
25         account = null;
26     }
27
28     @Test
29     void testDeposit() {
30         /** 2. A deposit $50 and check that the balance is 150 */
31         account.deposit( amount: 50.0);
32         assertEquals( expected: 150.0, account.getBalance());
33     }
34
12 class BankAccountTest {
35     @Test
36     void testWithdraw() {
37         /** 3. withdraw $40 and check that the balance is $60; remember that each test is done on a fresh instance of bank account */
38         account.withdraw( amount: 40.0);
39         assertEquals( expected: 60.0, account.getBalance());
40     }
41
42     @Test
43     void testInvalidDeposit() {
44         /** 4. Deposit a negative amount and check if an exception is thrown */
45         assertThrows(IllegalArgumentException.class, () -> account.deposit( amount: -50.0));
46     }
47
48     @Test
49     void testOverdraft() {
50         /** 5. Verify that Withdrawing more than the current balance
51         throws an exception */
52         assertThrows(IllegalArgumentException.class, () -> account.withdraw( amount: 200));
53     }
54
55     @Test
56     void testNegativeBalance() {
57         /** 6. Add a test to check that an Exception is thrown when
58         trying to create a new bankaccount with a negative initial balance */
59         BankAccount account2;
60         assertThrows(IllegalArgumentException.class, () -> account = new BankAccount( initialBalance: -100.0));
61     }
```

Output



Maven dependencies:



2.

Write a JUnit test assuming you have an array of int values and you only want the JUnit test to fail if any of the values are less than 20

```
public static int[] makeArray(){ no usages
    int[] arr = new Random().ints( streamSize: 10, randomNumberOrigin: 0, randomNumberBound: 20).toArray();
    return arr;
}
```

ed > src > main > java > org > example > Main 22:2 LF UTF-8 4 spaces

3.

Write a JUnit test assuming you have two String references named strOne and strTwo and you only want it to pass if the two Strings contain the same characters

The screenshot displays an IDE with the following components:

- Editor:** Shows the `MainTest.java` file with the following code:

```
12 public class MainTest {
22     @Test
23     public void testStrings(){
24         String[] arr = makeArrayString();
25         String strOne = arr[0];
26         String strTwo = arr[1];
27         char[] arrTwo = strOne.toCharArray();
28         char[] arrOne = strTwo.toCharArray();
29         Arrays.sort(arrOne);
30         Arrays.sort(arrTwo);
31
32         assertEquals(Arrays.toString(arrOne), Arrays.toString(arrTwo));
33     }
34 }
35
```
- Run Console:** Shows the execution of `MainTest` with the following output:

```
Run MainTest x
C:\Users\PHOENIX\jdk\openjdk-22\bin\java.exe ...
Process finished with exit code 0
```
- Test Results:** A table showing the results of the tests:

Test	Duration	Status
testArray()	19 ms	Passed
testStrings()	3 ms	Passed
- Snipping Tool:** A notification box in the bottom right corner stating: "Screenshot copied to clipboard. Automatically saved to screenshots folder. Markup and share".

4.

If you have three JUnit test methods written in the same testing class and the first one fails its assertions, will the other methods still be executed? (try it)

Yes the other test will run separately from the failed test

The screenshot shows an IDE with the following components:

- Editor:** Displays the `BankAccountTest_Template.java` file. The code includes:
 - `testDeposit()`: Deposits 50.0 and asserts the balance is 150.0.
 - `testWithdraw()`: Withdraws 40.0 and asserts the balance is 50.0. This test is marked as failed.
 - `testInvalidDeposit()`: Attempts to deposit a negative amount (-50.0) and asserts an `IllegalArgumentException` is thrown.
- Run Console:** Shows the execution of `BankAccountTest`. The summary indicates "Tests failed: 1, passed: 5 of 6 tests - 32 ms". The failed test is `testWithdraw`.
- Stack Trace:** The error for `testWithdraw` is an `org.opentest4j.AssertionFailedError` with the message "Expected :50.0 Actual :60.0". The stack trace points to line 39 in `BankAccountTest_Template.java`.

5.

Consider the class `Computations.java`. Write JUnit tests to verify that the methods work for a variety of values including edge cases

```
5. 9 public class ComputationsTest {
10     @Test
11     public void testFibonacci(){
12         int temp = Computations.fibonacci(n: 5);
13         assertEquals( expected: Computations.fibonacci(n: temp-1)+Computations.fibonacci(n: temp-2), temp);
14     }
15
16     @Test
17     public void testPrime(){
18         assertTrue(Computations.isPrime(n: 5));
19         assertFalse(Computations.isPrime(n: 6));
20     }
21
22     @Test
23     public void testEven(){
24         assertTrue(Computations.isEven(number: 2));
25         assertFalse(Computations.isEven(number: 3));
26     }
27
28     @Test
29     public void testOdd(){
30         assertTrue(Computations.isOdd(number: 3));
31         assertFalse(Computations.isOdd(number: 4));
32     }
33
34     @Test
35     public void testCelcius(){
36         int temp = 10;
37         assertEquals(((temp - 32) * 5.0 / 9.0), Computations.toCelsius(temp));
```

```
Current File v [play] [bug] [search] [share] [refresh] [close]
Main.java MainTest.java Computations.java ComputationsTest.java x BankAccount.java BankAccount.java v [notifications] [search] [share] [refresh] [close]
15. 9 public class ComputationsTest {
25         assertFalse(Computations.isEven(number: 3));
26     }
27
28     @Test
29     public void testOdd(){
30         assertTrue(Computations.isOdd(number: 3));
31         assertFalse(Computations.isOdd(number: 4));
32     }
33
34     @Test
35     public void testCelcius(){
36         int temp = 10;
37         assertEquals(((temp - 32) * 5.0 / 9.0), Computations.toCelsius(temp));
38     }
39
40     @Test
41     public void testFahrenheit(){
42         int temp = 10;
43         assertEquals(((temp * 9.0 / 5.0) + 32), Computations.toFahrenheit(temp));
44     }
45 }
46
47 }
```

